

API-Referenz

Vollständige Referenz aller öffentlichen Typen des Geef.Sdk (v1.0.0).

Einstiegspunkt

Geef (statische Klasse)

```
namespace Geef.Sdk;

public static class Geef
{
    public static GeefPipelineBuilder<TOutput> CreatePipeline<TOutput>();
}
```

Erstellt einen neuen Pipeline-Builder für den angegebenen Ausgabebetyp.

Builder und Runner

GeefPipelineBuilder<TOutput>

Fluent Builder zur Pipeline-Konfiguration. Trennt Definition (Builder) von Ausführung (Runner).

```

public sealed class GeefPipelineBuilder<TOutput>
{
    // Pflicht-Komponenten
    GeefPipelineBuilder<TOutput> UseGrounding(IGroundingStep grounding);
    GeefPipelineBuilder<TOutput> UseExecution(IExecutionStep execution);
    GeefPipelineBuilder<TOutput> AddReviewer(IRewriter reviewer);
    GeefPipelineBuilder<TOutput> UseFinalizer(IFinalizer<TOutput> finalizer);

    // Optionale Konfiguration
    GeefPipelineBuilder<TOutput> UseConvergencePolicy(IConvergencePolicy policy);
    GeefPipelineBuilder<TOutput> UseEvaluationStrategy(IEvaluationStrategy strategy);
    GeefPipelineBuilder<TOutput> UseMiddleware(IGeefMiddleware middleware);
    GeefPipelineBuilder<TOutput> UseMiddleware<TMiddleware>()
        where TMiddleware : IGeefMiddleware, new();
    GeefPipelineBuilder<TOutput> AddEventSink(IGeefEventSink sink);
    GeefPipelineBuilder<TOutput> ConfigureEvents(Action<DelegateEventSink> configure);

    // Build
    GeefPipelineRunner<TOutput> Build();
}

```

Build() validiert die Konfiguration und wirft **PipelineConfigurationException**, wenn eine Pflicht-Komponente fehlt:

- Grounding, Execution, Finalizer müssen gesetzt sein
- Mindestens ein Reviewer muss registriert sein

Defaults bei optionalen Komponenten:

- Convergence Policy: **DefaultConvergencePolicy** (10 Iterationen, 30 min)
- Evaluation Strategy: **SequentialEvaluationStrategy**
- Middleware: keine
- Event Sinks: **NullEventSink**

GeefPipelineRunner<TOutput>

Der Orchestrator. Führt den GEEF-Loop aus. Immutabel nach **Build()** und thread-safe — kann für mehrere gleichzeitige Runs wiederverwendet werden.

```
public sealed class GeefPipelineRunner<TOutput>
{
    Task<GeefPipelineResult<TOutput>> RunAsync(
        string input,
        CancellationToken cancellationToken = default);
}
```

Exceptions:

- `ProviderException` — Infrastruktur-Fehler einer Phase (Grounding, Execution, Evaluation, Finalize)
- `ConvergenceFailedException` — Konvergenz-Policy liefert terminale Entscheidung (MaxAttempts, Stagnation, Regression, CriticalBlocker, EscalateToHuman)
- `PhaseTimeoutException` — Eine Phase hat ihr Timeout überschritten (via `TimeoutMiddleware`)
- `OperationCanceledException` — Externes Cancellation-Signal

`GeefPipelineResult<TOutput>`

```
public sealed record GeefPipelineResult<TOutput>
{
    TOutput Output { get; init; }
    string RunId { get; init; }
    int TotalIterations { get; init; }
    TimeSpan TotalDuration { get; init; }
    bool Success { get; init; }
    IRunContext FinalContext { get; init; }
    IterationHistory History { get; init; }
}
```

Context

`ContextKey<T>`

```
public sealed record ContextKey<T>(string Name);
```

Typisierter Schlüssel für den Context-Store. Generischer Typ `T` erzwingt Typsicherheit zur Kompilierzeit.

IRunContext

```
public interface IRunContext
{
    T GetRequired<T>(ContextKey<T> key);
    bool TryGet<T>(ContextKey<T> key, out T? value);
    bool Contains<T>(ContextKey<T> key);
    IRunContext Set<T>(ContextKey<T> key, T value);
    IRunContext Remove<T>(ContextKey<T> key);
    IReadOnlyCollection<string> Keys { get; }
}
```

Immutabilität: `Set()` und `Remove()` geben eine **neue** Instanz zurück. Die ursprüngliche Instanz wird nie verändert.

RunContext

Die Standard-Implementierung von `IRunContext`, basierend auf `ImmutableDictionary<string, object>`. Thread-safe für beliebig viele gleichzeitige Leser.

GeefKeys (vordefinierte Schlüssel)

```
public static class GeefKeys
{
    public static readonly ContextKey<string> OriginalInput = new("geef:original-input");
    public static readonly ContextKey<IReadOnlyList<Finding>> PreviousFindings = new("geef:previous-findings");
    public static readonly ContextKey<int> CurrentIteration = new("geef:current-iteration");
    public static readonly ContextKey<DateTimeOffset> RunStartedAt = new("geef:run-started-at");
    public static readonly ContextKey<string> RunId = new("geef:run-id");
    public static readonly ContextKey<IterationHistory> IterationHistory = new("geef:iteration-history");
}
```

Provider-Interfaces

IGroundingStep

```
public interface IGroundingStep
{
    Task<GroundingResult> RunAsync(string input, CancellationToken ct = default);
}
```

IExecutionStep

```
public interface IExecutionStep
{
    Task<ExecutionResult> RunAsync(IRunContext context, CancellationToken ct = default);
}
```

IReviewer

```
public interface IReviewer
{
    string Name { get; }
    int Priority { get => 100; }
    Task<ReviewResult> ReviewAsync(IRunContext context, CancellationToken ct = default);
}
```

IFinalizer<TOutput>

```
public interface IFinalizer<TOutput>
{
    Task<FinalizeResult<TOutput>> FinalizeAsync(IRunContext context, CancellationToken ct = default);
}
```

Result-Typen

GroundingResult

```
public sealed record GroundingResult
{
    IRunContext Context { get; init; }
    IReadOnlyList<string> Notes { get; init; } // Default: []
}
```

ExecutionResult

```
public sealed record ExecutionResult
{
    IRunContext UpdatedContext { get; init; }
    IReadOnlyList<string> Notes { get; init; } // Default: []
}
```

ReviewResult

```
public sealed record ReviewResult
{
    string ReviewerName { get; init; }
    ReviewDecision Decision { get; init; }
    IReadOnlyList<Finding> Findings { get; init; } // Default: []
    TimeSpan Duration { get; init; }
    double? Confidence { get; init; }
    string? SuggestedRetryHint { get; init; }
}
```

FinalizeResult<TOutput>

```
public sealed record FinalizeResult<TOutput>
{
    TOutput Output { get; init; }
    IRunContext FinalContext { get; init; }
    string? Summary { get; init; }
}
```

EvaluationAggregate

```
public sealed record EvaluationAggregate
{
    IReadOnlyList<ReviewResult> Reviews { get; init; }
    IReadOnlyList<Finding> AllFindings { get; }           // Computed
    bool HasBlockingIssues { get; }                       // Computed
    bool IsFullyApproved { get; }                         // Computed
    TimeSpan TotalDuration { get; }                       // Computed
}
```

Finding

```
public sealed record Finding
{
    string ReviewerName { get; init; }
    string Fingerprint { get; init; }
    string Message { get; init; }
    FindingSeverity Severity { get; init; } // Default: Error
    string? Category { get; init; }
    string? ArtifactReference { get; init; }
    IReadOnlyDictionary<string, object>? Metadata { get; init; }
}
```

Enums

ReviewDecision

Wert	Beschreibung
Approved	Keine Probleme. Artefakte akzeptabel.
Rejected	Probleme gefunden. Erneute Execution nötig.
ApprovedWithWarnings	Akzeptiert, aber mit Warnungen.
RetrySuggested	Retry vorgeschlagen, ohne klares Problem.
NotApplicable	Reviewer nicht anwendbar.
Failed	Technischer Fehler des Reviewers.

FindingSeverity

Wert	Blockiert?	Beschreibung
Info	Nein	Informativ
Warning	Nein	Warnung
Error	Ja	Fehler, Loop iteriert
Critical	Sofort-Abbruch	Sicherheitskritisch

ConvergenceDecision

Wert	Beschreibung
Approved	Loop endet, Finalize beginnt
Continue	Weiter iterieren
StopMaxAttemptsReached	Max-Iterationen/Zeit erreicht
StopStagnant	Stagnation erkannt
StopRegression	Regression erkannt
AbortCriticalBlocker	Kritischer Blocker
EscalateToHuman	Eskalation an Menschen

GeefPhase

Wert	Beschreibung
Grounding	Kontextsammlung
Execution	Artefakterzeugung
Evaluation	Prüfung
Finalize	Finalisierung

Policies

IConvergencePolicy

```
public interface IConvergencePolicy
{
    ConvergenceDecision Evaluate(
        IterationHistory history,
        EvaluationAggregate currentAggregate,
        TimeSpan elapsed);
}
```

DefaultConvergencePolicy

```
public sealed class DefaultConvergencePolicy : IConvergencePolicy
{
    int MaxIterations { get; init; } // Default: 10
    TimeSpan MaxElapsedTime { get; init; } // Default: 30 min
    int StagnationThreshold { get; init; } // Default: 3
    bool AbortOnCritical { get; init; } // Default: true
    bool DetectRegression { get; init; } // Default: true
}
```

IEvaluationStrategy

```
public interface IEvaluationStrategy
{
    Task<EvaluationAggregate> ExecuteAsync(
        IReadOnlyList<IReviewer> reviewers,
        IRunContext context,
        CancellationToken ct = default);
}
```

Implementierungen: `SequentialEvaluationStrategy`, `ParallelEvaluationStrategy`, `FailFastEvaluationStrategy`, `PriorityOrderedEvaluationStrategy`

Middleware

IGeefMiddleware

```
public interface IGeefMiddleware
{
    Task InvokeAsync(GeefMiddlewareContext context, Func<Task> next);
}
```

GeefMiddlewareContext

```
public sealed class GeefMiddlewareContext
{
    GeefPhase Phase { get; init; }
    IRunContext RunContext { get; init; }
    string? ComponentName { get; init; }
    int? Iteration { get; init; }
    string RunId { get; init; }
    CancellationToken CancellationToken { get; set; }
    IDictionary<string, object> Properties { get; }
}
```

TimeoutMiddleware

```
public sealed class TimeoutMiddleware : IGeefMiddleware
{
    TimeSpan DefaultTimeout { get; init; } // Default: 5 min
    Dictionary<GeefPhase, TimeSpan> PhaseTimeouts { get; init; }
}
```

TracingMiddleware

Erstellt OpenTelemetry-Spans (`Activity`) für jede Phase mit Tags: `geef.phase`, `geef.run_id`, `geef.iteration`.

ExceptionHandlerMiddleware

Fängt unerwartete Exceptions und konvertiert sie in `ProviderException`. Setzt `Phase` aus dem Middleware-Context und `ProviderName` aus `GeefMiddlewareContext.ComponentName`. Hinweis: Der Standard-Runner fängt Provider-Exceptions bereits direkt ab und setzt `ProviderName` selbst — `Except`

`ionHandlingMiddleware` greift nur bei Exceptions, die nicht vom Runner abgefangen werden. In diesem Fall ist `ComponentName` ggf. `null`.

Events

IGeefEventSink

```
public interface IGeefEventSink
{
    ValueTask PublishAsync(IGeefEvent geefEvent, CancellationToken ct = default);
}
```

Event-Typen

Event	Zeitpunkt
<code>PipelineStartedEvent</code>	Pipeline-Lauf startet
<code>GroundingStartedEvent</code>	Grounding-Phase startet
<code>GroundingCompletedEvent</code>	Grounding-Phase endet
<code>ExecutionStartedEvent</code>	Execution-Phase startet
<code>ExecutionCompletedEvent</code>	Execution-Phase endet
<code>ReviewerStartedEvent</code>	Einzelner Reviewer startet
<code>ReviewerCompletedEvent</code>	Einzelner Reviewer endet
<code>EvaluationApprovedEvent</code>	Evaluationsrunde: genehmigt
<code>EvaluationRejectedEvent</code>	Evaluationsrunde: abgelehnt
<code>FinalizeStartedEvent</code>	Finalize-Phase startet
<code>FinalizeCompletedEvent</code>	Finalize-Phase endet
<code>PipelineCompletedEvent</code>	Pipeline-Lauf endet (Erfolg)
<code>PipelineFailedEvent</code>	Pipeline-Lauf endet (Fehler)

Event-Sink-Implementierungen

Klasse	Beschreibung
<code>NullEventSink</code>	No-Op. Standard wenn nichts konfiguriert.
<code>DelegateEventSink</code>	Delegate-Hooks für 11 von 13 Event-Typen (s.u.)
<code>CompositeEventSink</code>	Verteilt an mehrere Sinks sequentiell
<code>LoggingEventSink</code>	Loggt Events über <code>ILogger</code>

Hinweis: `DelegateEventSink` bietet Hooks für 11 der 13 Event-Typen. `ReviewerStartedEvent` und `ReviewerCompletedEvent` werden von `DelegateEventSink` nicht unterstützt — diese Events sind nur über eine eigene `IGeefEventSink`-Implementierung zugänglich.

Runtime

`IterationHistory`

```
public sealed class IterationHistory
{
    IReadOnlyList<IterationRecord> Records { get; }
    int Count { get; }
    TimeSpan TotalElapsed { get; }

    void Add(IterationRecord record);
    bool IsStagnant(int lookbackIterations = 3);
    bool HasRegression();
}
```

`IterationRecord`

```
public sealed record IterationRecord
{
    int Iteration { get; init; }
    DateTimeOffset StartedAt { get; init; }
    TimeSpan ExecutionDuration { get; init; }
    EvaluationAggregate EvaluationResult { get; init; }
    IReadOnlySet<string> FindingFingerprints { get; } // Computed
}
```

Exceptions

GeefException

```
public class GeefException : Exception
{
    string? RunId { get; init; }
}
```

ConvergenceFailedException

```
public sealed class ConvergenceFailedException : GeefException
{
    ConvergenceDecision Reason { get; init; }
    IterationHistory History { get; init; }
    EvaluationAggregate LastEvaluation { get; init; }
}
```

PhaseTimeoutException

```
public sealed class PhaseTimeoutException : GeefException
{
    GeefPhase Phase { get; init; }
    TimeSpan Timeout { get; init; }
}
```

ProviderException

```
public sealed class ProviderException : GeefException
{
    GeefPhase Phase { get; init; }
    string? ProviderName { get; init; }
}
```

Bei Reviewer-Fehlern wird `ProviderName` auf den Typ-Namen der `IEvaluationStrategy` gesetzt (z.B. `"SequentialEvaluationStrategy"`), nicht auf den individuellen Reviewer-Namen.

PipelineConfigurationException

```
public sealed class PipelineConfigurationException : GeefException { }
```

Alle GEEF-Exceptions tragen ein `RunId`-Property zur Korrelation.

Hosting / DI

GeefServiceCollectionExtensions

```
public static class GeefServiceCollectionExtensions
{
    static IServiceCollection AddGeefPipeline<TOutput>(
        this IServiceCollection services,
        Action<GeefPipelineBuilder<TOutput>> configure);

    static IServiceCollection AddGeefPipeline<TOutput>(
        this IServiceCollection services,
        Action<IServiceProvider, GeefPipelineBuilder<TOutput>> configure);
}
```

Diagnostics

GeefDiagnostics

```
public static class GeefDiagnostics
{
    static readonly ActivitySource ActivitySource; // Name: "Geef.Sdk", Version: "1.0.0"
}
```